

Programming Project II

Delivery Truck Pallet Packing Optimization

Desenho de Algoritmos 2024/2025

Table of Contents

1. Objectives and Problem Motivation
2. Project's code Structure and Features
3. All Algorithmic Approaches
 - 3.1. Brute Force Algorithm
 - 3.2. Dynamic Programming
 - 3.3. Greedy Algorithm
 - 3.4. Integer Linear Programming
4. Results Discussion and Data Analysis
5. Conclusion

Objectives and Problem Motivation

Objectives :

- Develop teamwork and project management skills
- implement 4 algorithmic approaches to solve the 0/1 Knapsack Problem for pallet loading
- Compare trade-offs between optimality and efficiency

Problem Motivation:

- Real-world logistics: Maximize the value of each shipment while adhering to weight
- Modeled as 0/1 Knapsack:
 - Truck = Knapsack (capacity W)
 - Pallets = Items (weight w , profit p)

Project's code Structure and Features

```
class Truck {  
    private:  
        int maxWeight = -1;  
        int currProfit = -1;  
        int totalPalletsNum = -1;  
        vector<Pallet> currPallets;  
    public:  
        //getters and setters  
};
```

```
class Pallet {  
    private:  
        bool inTruck;  
        int palletId;  
        int palletWeight;  
        int palletValue;  
    public:  
        //getters and setters  
};
```



```
src  
├── batchLogic.cpp  
├── batchLogic.h  
├── bruteForceAlg.cpp  
├── bruteForceAlg.h  
├── dynamicProgAlg.cpp  
├── dynamicProgAlg.h  
├── greedyAlg.cpp  
├── greedyAlg.h  
├── ILPAlg.cpp  
├── ILPAlg.h  
├── main.cpp  
├── menuLogic.cpp  
├── menuLogic.h  
├── output.txt  
├── pallet.h  
├── parseDataset.cpp  
├── parseDataSet.h  
└── truck.h
```

Project's code Structure and Features

Menu Mode

`./ Project02`

Batch Mode

`./ Project02 <int> DataSetNum <int> AlgorithmApproach`

<1 – 10>

TrunckAndPalletsX.csv
PalletsX.csv

1. **Brute Force**
2. **Dynamic Programming**
3. **Greedy Algorithm**
4. **ILP**

```
(Interactive Mode) ./Project02
(Batch Mode)      ./Project02 (int)DataSetNumber (int)AlgorithmNumber
```

> DataSetNumber (1-10): Selects the dataset pair to load

Value	Dataset Files
1	TrunkAndPallets1.csv, Pallets1.csv
2	TrunkAndPallets2.csv, Pallets2.csv
3	TrunkAndPallets3.csv, Pallets3.csv
4	TrunkAndPallets4.csv, Pallets4.csv
5	TrunkAndPallets5.csv, Pallets5.csv
6	TrunkAndPallets6.csv, Pallets6.csv
7	TrunkAndPallets7.csv, Pallets7.csv
8	TrunkAndPallets8.csv, Pallets8.csv
9	TrunkAndPallets9.csv, Pallets9.csv
10	TrunkAndPallets10.csv, Pallets10.csv

> AlgorithmNumber (1-4): Selects the algorithm to use

Value	Algorithm	Description
1	Brute Force	Tries all combinations
2	Dynamic Programming	Bottom-up approach for optimization
3	Greedy	Heuristic based on value/weight
4	ILP	Integer Linear Programming

Brute Force Algorithm

```
vector<Pallet> BruteForceAlgorithm(Truck& truck, vector<Pallet>& pallets) {  
    const int maxWeight = truck.getMaxWeight();  
    const int numPallets = truck.getTotalPalletsNum();  
    int maxValue = 0;  
    vector<Pallet> optSolutionList;  
  
    // iterate over all possible subsets (2^n)  
    for (int mask = 0; mask < (1 << numPallets); ++mask) {  
        int currWeight = 0;  
        int currValue = 0;  
        vector<Pallet> currSolution;  
  
        for (int i = 0; i < numPallets; ++i) {  
            if (mask & (1 << i)) {  
                currWeight += pallets[i].getPalletWeight();  
                currValue += pallets[i].getPalletValue();  
                currSolution.push_back(pallets[i]);  
            }  
        }  
  
        // Check if the subset is valid and optimal  
        if (currWeight <= maxWeight) {  
            if (currValue > maxValue) {  
                maxValue = currValue;  
                optSolutionList = currSolution; // store the best curr solution  
            }  
        }  
    }  
  
    return optSolutionList;  
}
```

Check All possible Pallet combinations

Example : n = 3

Compute the weight and profit of the current selection

Fetch Valid and Optimal Solution from all possible combos

<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>
<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>
<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>
<u>0</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>

0 1 1

Dynamic Programming

```
// --- Dynamic Programming Algorithm Structure --- //
```

```
    // Create n*W dp table to hold optimal subproblems solutions
```

```
    // Populate Table with most optimal assortment of pallets  
per cell
```

```
    // backtrack through dp table to fetch pallet assortment for  
optimal solution
```

//Create DP table

```
// init dp table
vector<vector<pair<int, pair<int, int>>>>
    dp(n + 1, value:vector<pair<int, pair<int, int>>>(n:W + 1, value: {a:0, b:{x:0, y:0}}));
```

Creates a (numberOfPallets+1 * MaxW + 1) 2D table

Example : 2 Pallets with 3 MaxW

```
dp = [
  [ {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}} ],
  [ {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}} ],
  [ {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}}, {0, {0, 0}} ]
]
```

{0, {0, 0}}

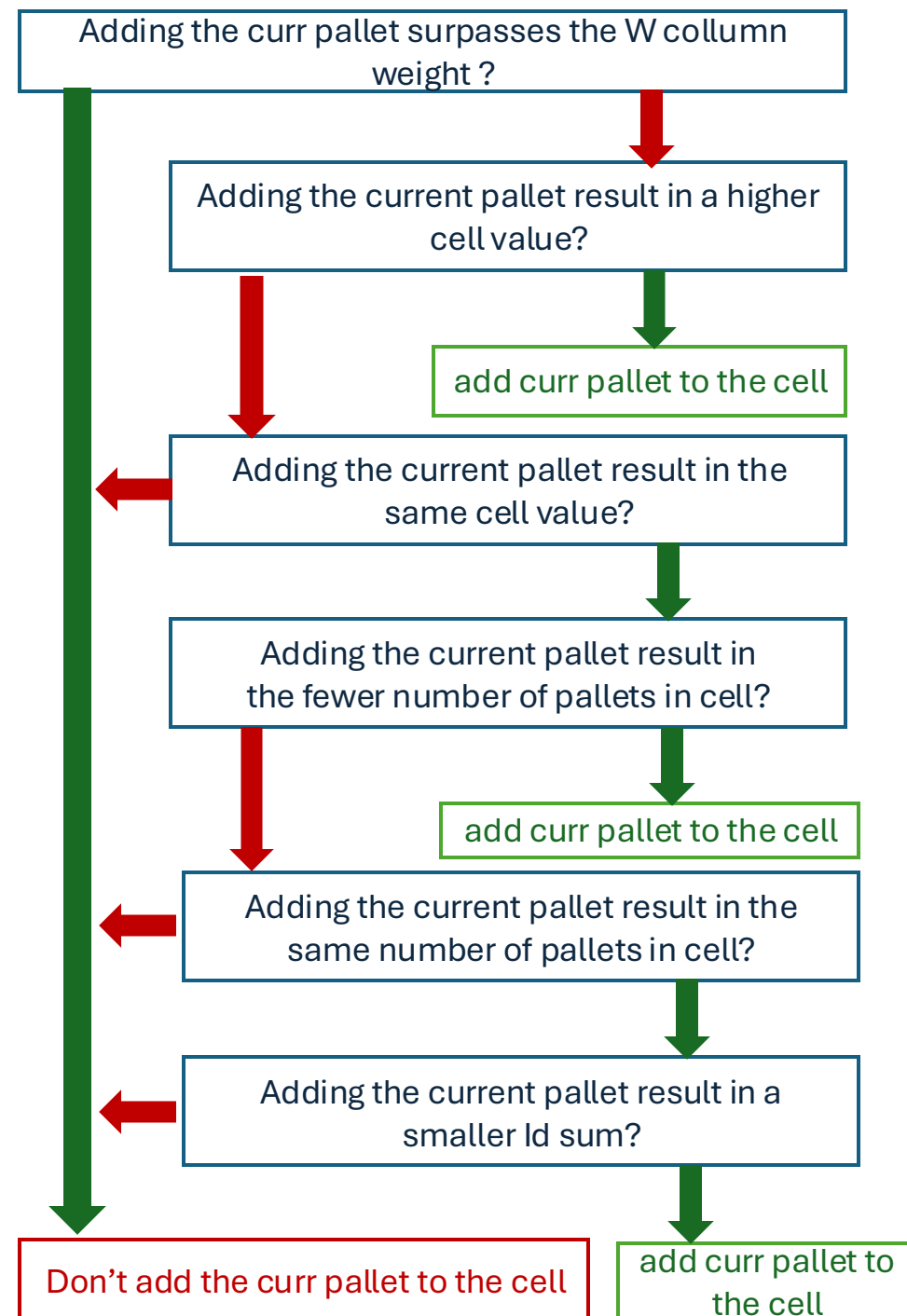
Sum of the ids of the pallets in this cell

Number of pallets in this cell

Total value of selected pallets

//Populate DP table

```
// populate dp table
for (int i = 1; i <= n; i++) {
    // fill row
    for (int w = 0; w <= W; w++) {
        // pallet too heavy
        if (currw > w)
            dp[i][w] = dp[i - 1][w]; //dont include pallet in dp cell
        else {
            // if higher value
            if (valueWithCurrent > valueWithoutCurrent)
                dp[i][w] = {valueWithCurrent, b:{x: [&palletsWithCurrent], y: [&idSumWithCurrent]}};
            // if equal value
            else if (valueWithCurrent == valueWithoutCurrent) {
                //take sol with fewer pallets
                if (palletsWithCurrent < palletsWithoutCurrent)
                    dp[i][w] = {valueWithCurrent, b:{x: [&palletsWithCurrent], y: [&idSumWithCurrent]}};
                //if same number of pallets
                else if (palletsWithCurrent == palletsWithoutCurrent) {
                    // take the one with smaller ids (smaller id sum)
                    if (idSumWithCurrent < idSumWithoutCurrent)
                        dp[i][w] = {valueWithCurrent, b:{x: [&palletsWithCurrent], y: [&idSumWithCurrent]}};
                    else
                        dp[i][w] = {valueWithoutCurrent, b:{x: [&palletsWithoutCurrent], y: [&idSumWithoutCurrent]}};
                }
            }
            else
                dp[i][w] = {valueWithoutCurrent, b:{x: [&palletsWithoutCurrent], y: [&idSumWithoutCurrent]}};
        }
        // if lower value take other previous sol
        else
            dp[i][w] = {valueWithoutCurrent, b:{x: [&palletsWithoutCurrent], y: [&idSumWithoutCurrent]}};
    }
}
```



//Backtrack DP table

Start searching at dp[n][W]

```
// backtrack dp table
vector<Pallet> res;
int remainingW = W;

for (int i = n; i > 0 && remainingW > 0; i--) {
    // current pallet info
    int currw = weights[i - 1];
    int currv = values[i - 1];
    int currId = ids[i - 1];

    // check if pallet is part of solution
    if (currw <= remainingW &&
        dp[i][remainingW].first == dp[i - 1][remainingW - currw].first + currv &&
        dp[i][remainingW].second.first == dp[i - 1][remainingW - currw].second.first + 1 &&
        dp[i][remainingW].second.second == dp[i - 1][remainingW - currw].second.second + currId) {
        res.push_back(pallets[i - 1]);

        remainingW -= currw;
    }
}
```

Is the remaining weight higher or equal to pallet's weight?

Did including the pallet increase total value?

Was pallets number in cell incremented by 1?

Was id sum incremented by curr pallet id?

add curr pallet to the result vector

Don't add the curr pallet to result vector

Greedy Algorithm

```
vector<Pallet> GreedyAlgorithm(Truck &truck, vector<Pallet> &pallet) {  
    vector<Pallet> sortedPallets = pallet;  
  
    const int maxWeight = truck.getMaxWeight();  
    vector<Pallet> selectedPallets;  
    int currWeight = 0;  
  
    //sort pallets by value-to-weight ratio (desc)  
    sort(sortedPallets.begin(), sortedPallets.end(), sortPallets);  
  
    //greedy select the best ratio pallets  
    for (const Pallet &p : sortedPallets) {  
        if (currWeight + p.getPalletWeight() <= maxWeight) {  
            selectedPallets.push_back(p);  
            currWeight += p.getPalletWeight();  
        }  
    }  
  
    return selectedPallets;  
}
```

Sort Descending by the
Profit/Weight Ratio

Keep Adding From the sorted
pallet list until the next pallet
doesn't fit the weight limit

Integer Linear Programming

For each Pallet, p_i , we have:

$$p_i \in \{0, 1\} \quad p_i = \begin{cases} 0 \Rightarrow \text{item } i \text{ is not selected} \\ 1 \Rightarrow \text{item } i \text{ is selected} \end{cases} \quad \begin{array}{l} v \leftarrow p \text{ (Value of the pallet)} \\ w \leftarrow p \text{ (Weight of the pallet)} \end{array}$$

The goal is to maximize the value of the set of items selected (Objective Function):

$$\max \sum_{i=1}^n v_i \leftarrow p_i$$

With the constraint that the weight of the elements cannot surpass the weight limit on the truck (Capacity Constraint):

$$\sum_{i=1}^n (w_i \leftarrow p_i) \leq \text{MaxWeight}$$

Integer Linear Programming

Chosen Method for the ILP => **Branch and Bound**

Why?

- ✓ Ensures Optimal Solution
- ✓ Can be further optimized for larger datasets
- ✓ Handles constraints naturally
- ✓ Clear Traceability and Backtracking

Why Not other Methods?

Hybrid Approach

- > usually sacrifice optimality for speed
- > complex implementation
- > hard to add constraints and tie breakers
- > high complexity

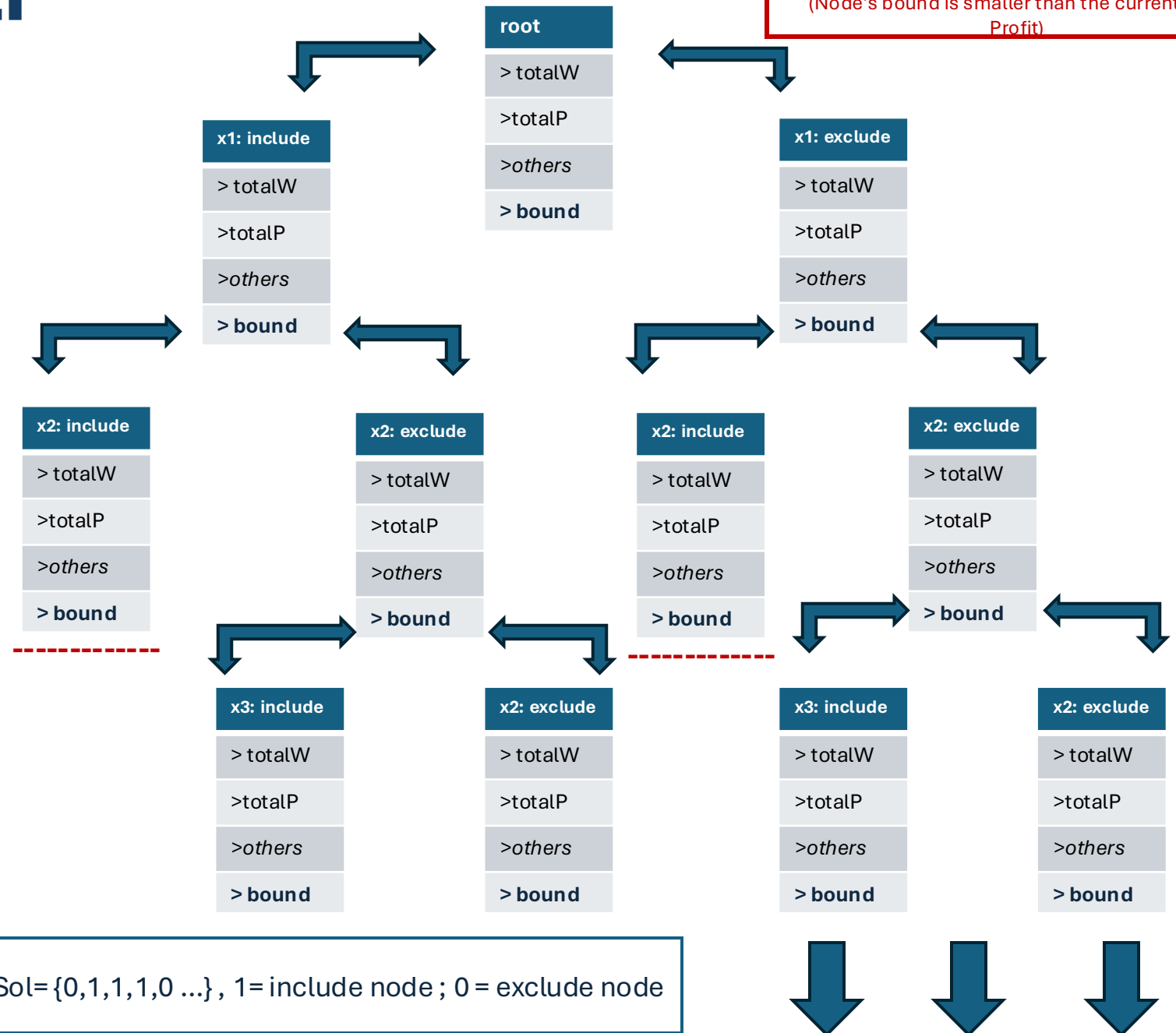
Cutting Planes

- > Linear Programming overhead
- > hard to add constraints and tie breakers
- > high complexity

ILP

Pruning: stop branch exploration

(Node's bound is smaller than the current total Profit)



```
while (!pq.empty()) {
    Node curr = pq.top();
    pq.pop();

    // prune if bound is lower than the current best total profit
    if (curr.bound < bestNode.totalProfit)
        continue;

    // if no more nodes to explore, update best node
    if (curr.level == n) {
        if (curr.totalWeight <= truck.getMaxWeight() && better(curr, bestNode))
            bestNode = curr;
        continue;
    }

    int i = curr.level;
    int w = pallets[i].getPalletWeight();
    int p = pallets[i].getPalletValue();
    int id = pallets[i].getPalletId();

    // scenario 1: Include pallet i
    if (curr.totalWeight + w <= truck.getMaxWeight()) {
        Node inc = curr;
        inc.level = i + 1;
        inc.totalWeight += w;
        inc.totalProfit += p;
        inc.palletCount += 1;
        inc.idSum += id;
        inc.path.push_back(i);
        inc.bound = bound(inc, truck, pallets, n);

        if (better(inc, bestNode))
            bestNode = inc;

        if (inc.bound >= bestNode.totalProfit)
            pq.push(inc);
    }

    // scenario 2 : dont include pallet i
    Node exc = curr;
    exc.level = i + 1;
    exc.bound = bound(exc, truck, pallets, n);

    // scenario 2 : dont include pallet i
    Node exc = curr;
    exc.level = i + 1;
    exc.bound = bound(exc, truck, pallets, n);

    if (exc.bound >= bestNode.totalProfit)
        pq.push(exc);
}
```

ILP - Aux

```
bool better(const Node& a, const Node& b) {
    if (a.totalProfit != b.totalProfit)
        return a.totalProfit > b.totalProfit;
    if (a.palletCount != b.palletCount)
        return a.palletCount < b.palletCount;
    return a.idSum < b.idSum;
}

double bound(const Node& node, const Truck& truck, const vector<Pallet>& pallets, int n) {
    if (node.totalWeight >= truck.getMaxWeight())
        return 0.0;

    double profitBound = node.totalProfit;
    int weight = node.totalWeight;

    for (int i = node.level; i < n; ++i) {
        int w = pallets[i].getPalletWeight();
        int p = pallets[i].getPalletValue();
        if (weight + w <= truck.getMaxWeight()) {
            weight += w;
            profitBound += p;
        } else {
            int remain = truck.getMaxWeight() - weight;
            profitBound += p * (double(remain) / w);
            break;
        }
    }
    return profitBound;
}
```

```
// sort by ratio and if tie sort by id
sort(first pallets.begin(), last pallets.end(), comp: [](auto& a, auto& b) {
    double r1 = double(a.getPalletValue()) / a.getPalletWeight();
    double r2 = double(b.getPalletValue()) / b.getPalletWeight();
    if (r1 != r2) return r1 > r2;
    return a.getPalletId() < b.getPalletId();
});
```

Sort is used to maximize pruning and algorithm efficiency, since we'd start with a very high bound, most assortments followed would be pruned at an early stage

better is an aux function that takes every optimal solution and makes sure only the one with the right constraints get returned

Bound is an aux function that calculates the max profit still achievable on a node, and, therefore, determines if a branch should keep getting searched or if it should get pruned

Results Analysis

We tested all the approaches against Optimal solutions given by the teacher and we concluded (apart from the runtime timeouts that are to be expected) that we designed correct algorithms that pass all tests.

algorithm	dataset	Matches Solution with optimal solution
Exhaustive Search	5	> runtime timeout (dataset too large)
	6	> runtime timeout (dataset too large)
	7	yes
	8	yes
	9	yes
	10	yes
DP	5	yes
	6	yes
	7	yes
	8	yes
	9	yes
	10	yes
Greedy	5	Not supposed to match opt sol
	6	
	7	
	8	
	9	
	10	
ILP	5	yes
	6	yes
	7	> runtime timeout (dataset too large)
	8	yes
	9	yes
	10	yes

Results Analysis

Algorithm	Worst Case	Average Case	Best Case	Space Complexity	Is it Optimal?
Brute Force	$O(2^n)$	$O(2^n)$	$O(2^n)$	$O(n)$	✓
Dynamic Programming	$O(n.W)$	$O(n.W)$	$O(n.W)$	$O(n.W)$	✓
Greedy Approach	$O(n.\log(n))$	$O(n.\log(n))$	$O(n.\log(n))$	$O(n)$	✗
ILP – Branch and Bound	$O(2^n)$	$O(2^k)$	$O(n.\log(n))$	$O(n)^*$	✓

n : number of pallets

W : max Truck weight capacity

k : effective node number (after pruning bad branches)

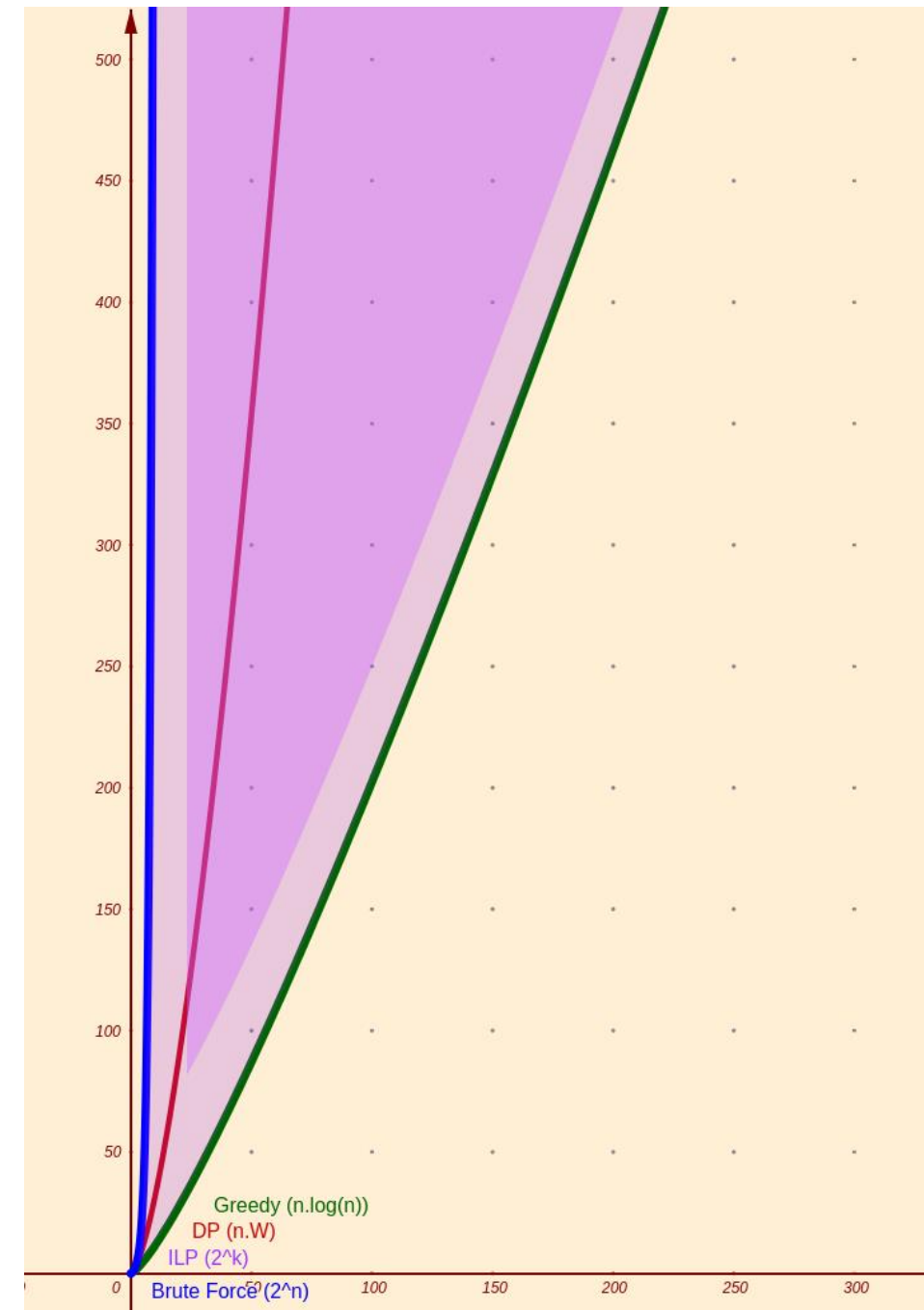
** plus queue overhead*

Results Analysis

The graph besides showcases the efficiency (purely timewise) of the each algorithm presented in this project

Although the "W" is a unrelated variable to the total number of pallets, for visual porpuses, we represented it as $\sqrt{n}$, since it we find representing the square root of said total number to be fitting for the Max Truck Capacity

And since the "k" element is highly deterministic to the efficiency of the ILP Branch and Bound algorithm, ILP is represented, in the graph, as an estimate (interval of values) ranging from its best Case time complexity scenario to its worst one – with a more realistic approximation, highlighted in a darker tone



When to use each algorithm approach (Conclusion)

Exhaustive Search (Brute Force)

=> Only on very small Datasets, due to its simple nature but exponential Time Complexity $O(2^n)$

Dynamic Programming

=> Small W and medium-large datasets, due to its pseudo-polynomial Time Complexity $O(nW)$ but High Space complexity (needs 2d table of size $n \times W$)

Greedy Approach

=> Very Large Data Sets when optimality isn't critical, due to it being an approach with polynomial Time Complexity, but may not produce optimal solutions

ILP (Branch and Bound)

=> Best for small-medium datasets with large W , aiming for an exact optimal solution while being more memory conservative than Dynamic Programming and more efficient than Brute Force